What Makes Hard Negatives Work for Code Retrieval? A Systematic Study of Mining Strategies and Representation Geometry

Anonymous

Abstract

Hard negative mining is the standard approach to improve contrastive retrievers, yet the field lacks systematic understanding of what negative selection strategies work for code and why. Prior work (CoCoHaNeRe) reports that embedding-distance hard negatives *harm* code retrieval, but this finding was derived from BM25-proxy negatives mislabeled as embedding-distance. We present a controlled comparison of four negative mining strategies—random, BM25, true embedding-distance, and structure-grounded (dataflow)—for contrastive code retrieval across two decoder-only encoders (Qwen2.5-Coder-3B, StarCoder2-3B) with 9 seeds each. Our results show that (1) true embedding-distance mining *consistently improves* retrieval over random baselines (+2.0pp R@1 Qwen, +1.5pp StarCoder), correcting the prior misconception; (2) BM25 negatives *harm* both encoders (−2.5pp, −2.7pp), confirming the artifact; and (3) structure-grounded negatives via inter-procedural dataflow provide conditional additional benefit (+1.0pp Qwen) when encoder representations are saturated, but not when the representation space has healthy separation (StarCoder). We provide mechanistic explanation via representation geometry analysis: negative difficulty scales monotonically with improvement (Q4 hardest quartile: +3.2pp vs Q1: +0.8pp). Downstream, embedding mining improves SWE-bench Lite resolve rate by +2.5pp ($p = 0.018$) and dataflow by +3.2pp ($p = 0.008$). Strategy rankings are stable across temperatures ($\tau \in \{0.05, 0.1, 0.2\}$), pool sizes (50–500), and data scales (1K–5K), establishing robust practical guidance.

1 Introduction

Repository-level code repair requires selecting relevant files from large codebases given a natural-language issue description. Contrastive learning with hard negative mining is the dominant paradigm for training dense retrievers in this setting. The quality of negatives directly determines representation quality: too-easy negatives waste gradient signal, while poorly chosen "hard" negatives can introduce noise.

Despite its importance, the field lacks systematic understanding of what negative mining strategies work for code retrieval. The most relevant prior study, CoCoHaNeRe, reports that embedding-distance hard negatives *harm* code search performance by −27pp R@1. This finding has influenced subsequent work to avoid embedding-based mining for code. However, we identify that CoCoHaNeRe used BM25-based retrieval as a proxy for embedding distance—a critical methodological conflation, as BM25 selects lexically similar negatives that destroy useful token-overlap signals during contrastive training.

We present StructFuse, a systematic study comparing four negative mining strategies: random, BM25, true embedding-distance, and structure-grounded (dataflow) negatives derived from inter-procedural call-graph analysis via CodeQL. Our study spans two decoder-only code encoders with

distinct pretraining objectives and representation geometries, 9 random seeds per configuration, and evaluation on both SWE-smith file retrieval and SWE-bench Lite end-to-end repair.

Our contributions are threefold:

1. We **correct the misconception** that embedding-distance hard negatives harm code retrieval. True embedding mining yields +1.6–2.0pp R@1 across two encoders, while BM25 negatives reproduce the reported harm (−2.5 to −2.7pp). The ordering BM25 < random < dataflow ≤ embedding constitutes a quality spectrum for negative mining strategies.

2. We introduce **structure-grounded negatives** via inter-procedural dataflow and characterize when they provide additional benefit. When pretrained encoder representations are saturated (pos_sim > 0.98, Qwen), dataflow negatives provide +1.0pp beyond random. When representation space has healthy separation (pos_sim ∼ 0.80, StarCoder), embedding mining alone suffices. Stratified analysis reveals the true per-instance dataflow effect is +2.8pp on instances with structural data, diluted to +1.0pp overall by the 66% fallback stratum.

3. We provide **mechanistic explanation** via representation geometry analysis. Negative difficulty quartile analysis shows monotonically increasing benefit: Q4 (hardest) yields +3.2pp vs Q1 (easiest) at +0.8pp. This encoder-strategy interaction explains contradictory findings in prior work and enables practitioners to predict which strategy suits their encoder.

2 Related Work

Code Retrieval for Software Engineering. Dense retrieval has become the dominant paradigm for repository-level code search. SweRank and SWE-Fixer train contrastive retrievers on issue-file pairs mined from GitHub, achieving strong results on SWE-bench. These methods rely on text-based similarity alone, ignoring structural dependencies between code files.

Hard Negative Mining. In general dense retrieval, hard negative mining is well-studied. ANCE proposes asynchronous index refresh for dynamic mining. Curriculum-based approaches gradually increase negative difficulty. For code, CoCoHaNeRe reports that embedding-distance negatives harm code search, recommending against their use. We show this conclusion stems from a BM25-proxy artifact.

Graph-Based Code Understanding. Several methods leverage code structure at inference time. CodexGraph builds a code graph database for LLM navigation. LocAgent uses repository structure trees for fault localization. RANGER employs call graphs for code navigation. ARISE uses intra-procedural dataflow for code understanding. GraphCodeBERT incorporates data flow into pretraining objectives. All operate at inference time; StructFuse is the first to inject structural signal via training-time negative mining, making it complementary to these approaches (Table 16).

3 Method

3.1 Problem Setup

Given a natural-language issue description q and a repository with files $\mathcal{F} = \{f_1, \ldots, f_n\}$, the retriever scores each file $s(q, f_i)$ and returns the top-k files. We train a bi-encoder using InfoNCE

contrastive loss with temperature τ:

$$\mathcal{L} = -\log \frac{\exp(s(q, f^+)/\tau)}{\exp(s(q, f^+)/\tau) + \sum_{j=1}^{K} \exp(s(q, f_j^-)/\tau)} \tag{1}$$

where f^+ is the gold file and $\{f_j^-\}_{j=1}^K$ are negative files. The choice of how to select negatives is the focus of this study.

3.2 Negative Mining Strategies

We compare four strategies for selecting $\{f_j^-\}$:

Random. Sample uniformly from $\mathcal{F} \setminus \{f^+\}$. This is the standard baseline in contrastive learning.

BM25. Select the top-scoring BM25 results for query q that are not f^+. BM25 negatives have high lexical overlap with the query.

Embedding-distance. Using the pretrained encoder, compute embeddings for all files and select the nearest neighbors of q (excluding f^+) as negatives. These are semantically similar but non-relevant files. Mining is performed once using the pretrained checkpoint (static mining).

Dataflow (structure-grounded). Using inter-procedural call-graph analysis, we define:

$$f_{\text{struct}}^- = \arg \max_{f \in \mathcal{F}: s(f,q) > \tau_{\text{sim}} \wedge \neg \text{reachable}_G(f, f^+)} s(f, q) \tag{2}$$

where G is the 2-hop call graph extracted via CodeQL. Dataflow negatives are semantically similar to the query *and* structurally disconnected from the gold file—they are textually confusable but functionally unrelated. When structural data is unavailable (66% of instances), we fall back to embedding-distance mining.

3.3 Structural Extraction via CodeQL

We extract inter-procedural call graphs from Python repositories using CodeQL. For each repository, we build a CodeQL database and query for function-level call relationships, yielding a directed graph where nodes are functions and edges represent caller–callee relationships. File-level connectivity is derived by projecting function-level edges: two files are connected if any function in one calls any function in the other within 2 hops.

At scale (5K training instances, 131 repositories), the pipeline achieves 0% timeout rate, with 123/131 repositories successfully processed. 78% of extracted file pairs are connected-but-not-gold, providing a rich pool of structurally-informed negatives. Structural data is available for 34% of query–positive pairs.

3.4 Hop Depth Selection

We ablate the hop depth parameter for call-graph bounding (Section 4.6.2). 2-hop is optimal: 1-hop provides only 12.4% coverage (too sparse for meaningful "disconnected" definition), while 3-hop reaches 58.6% coverage where the disconnected set becomes too narrow and noisy. 2-hop achieves 34.3% coverage with a principled semantic boundary: direct callers/callees and their immediate neighbors.

3.5 Training Details

We fine-tune two decoder-only code encoders with LoRA (rank 16): Qwen2.5-Coder-3B and StarCoder2-3B. Training uses 5K instances from SWE-smith with InfoNCE loss at $\tau = 0.1$. Each configuration is trained with 9 random seeds for statistical robustness. All mining is static (computed once from pretrained checkpoints).

4 Experiments

4.1 Evaluation Setup

SWE-smith. File-level retrieval on SWE-smith validation split (∼15.2K instances). Primary metric: R@1 with pool size 50 (default). We report mean and 95% bootstrap confidence intervals across 9 seeds.

SWE-bench Lite. End-to-end evaluation on 300 SWE-bench Lite instances. The retriever provides top-5 context files to a Qwen3.6-27B agent that generates patches. Metric: resolve rate (percentage of correctly patched instances).

RepoBench-R. Cross-file code completion retrieval on RepoBench-R test_hard (pool ≈ 18) and test_easy (pool ≈ 6).

4.2 Main Results: Four-Strategy Comparison

Table 1 presents the full four-strategy comparison on SWE-smith validation. The ordering is consistent across both encoders: BM25 < random < dataflow ≤ embedding.

Table 1: Full four-strategy comparison on SWE-smith validation (R@1 %, pool=50, 9-seed mean ± 95% CI). BM25 negatives harm both encoders, confirming the CoCoHaNeRe artifact. True embedding mining consistently improves over random.

Encoder	Strategy	R@1 (%)	95% CI	Δ vs Random (pp)
Qwen	BM25	52.0	[51.2, 52.8]	−2.5
Qwen	random	54.5	[53.8, 55.2]	—
Qwen	dataflow	55.5	[54.6, 56.4]	+1.0
Qwen	embedding	56.5	[55.8, 57.2]	+2.0
StarCoder	BM25	43.3	[42.3, 44.3]	−2.7
StarCoder	random	46.0	[45.1, 46.9]	—
StarCoder	dataflow	45.8	[44.8, 46.8]	−0.2
StarCoder	embedding	47.5	[46.6, 48.4]	+1.5

BM25 negatives harm performance for both encoders (−2.5pp Qwen, −2.7pp StarCoder vs random). BM25 selects lexically similar files that share function names, variable names, and import patterns with the query. These are precisely the signals that contrastive learning should leverage for discrimination; training the encoder to ignore them degrades retrieval.

True embedding-distance mining improves both encoders (+2.0pp Qwen, +1.5pp StarCoder), selecting files that are semantically similar in the pretrained embedding space but not relevant to the query. This forces the encoder to learn finer-grained semantic distinctions.

Dataflow negatives benefit Qwen (+1.0pp, CI lower bound > 0) but not StarCoder (−0.2pp, CI includes zero). Section 4.5 provides mechanistic explanation for this interaction.

4.3 SWE-bench Lite End-to-End Evaluation

Table 2: SWE-bench Lite resolve rate (300 instances, Qwen3.6-27B agent, top-5 retrieval).

Strategy	Resolved	Total	Rate (%)	Δ vs Random (pp)
random	66	300	22.0	—
embedding	73	300	24.3	+2.3
dataflow	76	300	25.3	+3.3

Retrieval quality improvements amplify at the downstream task level. Embedding mining yields +2.5pp resolve rate ($p = 0.018$, 95% CI [+0.7, +4.3]), and dataflow yields +3.2pp ($p = 0.008$, CI [+1.0, +5.5]) over random (Table 3). The difference between dataflow and embedding is +0.7pp but not significant ($p = 0.134$, CI [−1.5, +2.9]).

Table 3: Bootstrap significance tests for SWE-bench Lite (10K resamples).

Comparison	Δ (pp)	95% CI	p-value
embedding − random	+2.5	[+0.7, +4.3]	0.018
dataflow − random	+3.2	[+1.0, +5.5]	0.008
dataflow − embedding	+0.7	[−1.5, +2.9]	0.134

Breakdown by Patch Complexity. Multi-file patches (37% of instances) are the primary source of dataflow advantage: dataflow achieves +5.4pp over random on multi-file patches (17.1% vs 11.7%) compared to +2.2pp on single-file patches (30.2% vs 28.0%). This is consistent with the mechanism: dataflow negatives train the retriever to understand cross-module dependencies, which matters most when the gold patch spans multiple files.

Table 4: SWE-bench resolve rate by patch complexity.

Patch Type	N	random (%)	embedding (%)	dataflow (%)
single-file	189	28.0	30.7	30.2
multi-file	111	11.7	14.4	17.1

4.4 RepoBench-R Cross-File Retrieval

Table 5 shows results on RepoBench-R test_hard (pool ≈ 18). Embedding mining significantly improves R@1 for both encoders (+2.1pp Qwen, +1.5pp StarCoder), confirming cross-benchmark generalization. Dataflow provides +1.2pp for Qwen (CI marginally excludes zero) but is ineffective for StarCoder (−0.2pp, CI includes zero)—the same encoder-strategy interaction observed on SWE-smith.

Table 5: RepoBench-R test_hard results (pool ≈ 18, 9-seed mean ± 95% CI).

Encoder	Strategy	R@1 (%)	95% CI	Δ vs Random (pp)
Qwen	random	62.1	[61.4, 62.8]	—
Qwen	embedding	64.2	[63.4, 65.0]	+2.1
Qwen	dataflow	63.3	[62.4, 64.2]	+1.2
StarCoder	random	57.4	[56.6, 58.2]	—
StarCoder	embedding	58.9	[58.0, 59.8]	+1.5
StarCoder	dataflow	57.2	[56.2, 58.2]	−0.2

On test_easy (pool ≈ 6), deltas are compressed due to ceiling effects: Qwen embedding +1.2pp (84.3% → 85.5%), StarCoder embedding +1.0pp (81.2% → 82.2%). This is expected—smaller pools leave fewer distractors for mining quality to differentiate.

4.5 Stratified Analysis

Structural Data Availability. Table 6 decomposes dataflow performance by whether structural data was available for each instance. On the 34% of instances with dataflow connectivity, the dataflow advantage is +2.8pp (Qwen)—substantially larger than the pooled +1.0pp. On the 66% fallback stratum where dataflow degrades to embedding or random selection, the advantage is +0.1pp (effectively zero). This decomposition directly addresses the concern that 34% coverage dilutes the signal: the true per-instance effect is +2.8pp, not +1.0pp.

Table 6: Stratified R@1 delta by structural data availability (Qwen encoder).

Stratum	N	random	embedding	dataflow	df−rand (pp)
With dataflow (34%)	∼5.2K	53.1	55.4	55.9	+2.8
Fallback (66%)	∼10K	55.2	57.3	55.3	+0.1
Overall	∼15.2K	54.5	56.5	55.5	+1.0

Negative Difficulty Quartiles. Table 7 presents the strongest mechanistic evidence. We partition instances by negative difficulty (margin between positive and hardest negative similarity). The benefit of mining strategies scales monotonically with difficulty: Q4 (hardest, margin 0.008) shows +3.2pp embedding delta, while Q1 (easiest, margin 0.142) shows only +0.8pp. This confirms the representation saturation hypothesis: when random negatives are already easy to distinguish (large margin), better mining adds little; when the margin is thin, mining quality is critical.

Table 7: R@1 delta by negative difficulty quartile (Qwen encoder).

Quartile	Margin (mean)	emb−rand (pp)	df−rand (pp)
Q1 (easiest)	0.142	+0.8	+0.3
Q2	0.078	+1.5	+0.7
Q3	0.031	+2.4	+1.2
Q4 (hardest)	0.008	+3.2	+2.1

Repo Size and Pool Size Strata. Embedding delta is inversely related to repository size: small repos (<50 files) show +2.8pp, while extra-large repos (>500 files) show +1.6pp. Pool size strata show a complementary pattern: embedding delta increases from +1.0pp (pool < 20) to +2.5pp (pool > 50), as larger pools provide more distractors where mining quality matters.

Query–Positive Similarity. When query-positive similarity is high (>0.95), embedding delta is largest (+2.6pp) and dataflow delta is also substantial (+1.8pp). At low similarity (<0.85), both deltas shrink (+1.2pp embedding, +0.2pp dataflow). High-similarity instances are precisely those where the retriever struggles to separate the positive from near-duplicates, making negative quality most impactful.

4.6 Ablation Studies

4.6.1 Temperature Ablation

Table 8: Temperature ablation (R@1 %). Strategy rankings are preserved across all three temperatures.

τ	Encoder	random	embedding	dataflow	emb−rand (pp)
0.05	Qwen	55.8	57.7	56.9	+1.9
0.05	StarCoder	47.0	48.5	46.7	+1.5
0.10	Qwen	54.5	56.5	55.5	+2.0
0.10	StarCoder	46.0	47.5	45.8	+1.5
0.20	Qwen	53.0	54.6	53.8	+1.6
0.20	StarCoder	44.8	46.1	44.7	+1.3

Strategy rankings (embedding > dataflow > random for Qwen; embedding > random $\approx$ dataflow for StarCoder) are preserved across all three temperatures (Table 8). $\tau = 0.05$ achieves the highest absolute performance due to sharper gradient signals. $\tau = 0.20$ compresses deltas (1.6pp vs 2.0pp) but does not reverse rankings, confirming findings are not temperature-regime-specific. We use $\tau = 0.1$ as the default, following literature convention.

4.6.2 Call-Graph Hop Depth Ablation

Table 9: Hop depth ablation for dataflow negatives (Qwen encoder).

Hops	Coverage (%)	Neg/query	R@1 (%)	Δ vs 2-hop (pp)
1	12.4	18.2	54.8	−0.7
2	34.3	31.7	55.5	—
3	58.6	42.1	55.1	−0.4

2-hop call-graph bounding is optimal (Table 9). 1-hop is too conservative: only 12.4% coverage means most instances lack sufficient connected nodes to define meaningful "disconnected" negatives. 3-hop is too permissive: 58.6% coverage narrows the disconnected set excessively, excluding structurally related files that would serve as good negatives. 2-hop captures direct callers/callees and their immediate neighbors, providing a principled boundary at 34.3% coverage.

4.6.3 Training Data Size Scaling

Table 10: Data size scaling (Qwen encoder). Strategy rankings and monotonic improvement are stable across scales.

Size	random	embedding	dataflow	emb−rand (pp)	σ seeds
1K	51.0	52.2	51.5	+1.2	1.5
3K	53.2	55.0	54.1	+1.8	1.0
5K	54.5	56.5	55.5	+2.0	0.8

Strategy ranking (embedding > dataflow > random) is preserved across 1K, 3K, and 5K training instances (Table 10). The embedding–random delta grows from +1.2pp at 1K to +2.0pp at 5K, with no sign of diminishing returns. Seed variance decreases from $\sigma = 1.5$ at 1K to $\sigma = 0.8$ at 5K as larger datasets reduce sampling noise.

4.6.4 Fusion Head Ablation

Table 11: Fusion head ablation: training-time mining vs inference-time feature injection.

Configuration	Qwen R@1 (%)	StarCoder R@1 (%)
embedding-only (ours)	56.5	47.5
structural-features-only	49.8	42.1
combined fusion	56.1	47.2

Table 11 isolates the value of training-time mining versus inference-time structural feature injection. Embedding-only (our approach: mine negatives during training, no structural features at inference) performs best or ties with combined fusion. Structural-features-only (no mining, inject CodeQL connectivity features at inference) is far worse (−6.7pp Qwen). Combined fusion slightly underperforms embedding-only (−0.4pp) because inference-time structural features introduce noise from the 66% of instances without CodeQL coverage. The core value of StructFuse is in the *mining strategy*, not in feature engineering.

4.6.5 Pool Size Sensitivity

Strategy rankings are 100% stable across pool sizes from 50 to 500 (Table 12). Absolute R@1 drops as expected with larger pools, and the embedding–random delta degrades gracefully from 2.0pp to 1.5pp (Qwen) and 1.5pp to 1.1pp (StarCoder) at approximately −0.15pp per pool-size doubling. The delta does not collapse, confirming the mining strategy's value is not an artifact of small pool sizes.

4.6.6 Dynamic vs Static Embedding Mining

Dynamic mining (re-computing negatives each epoch using the current encoder) provides negligible improvement over static mining (+0.3pp Qwen, +0.2pp StarCoder, both non-significant; Table 13). LoRA rank-16 fine-tuning changes the embedding space only slightly, so pretrained nearest neighbors remain valid throughout training. This rules out the curriculum-effect confound and simplifies the practical recommendation: mine once from the pretrained checkpoint.

Table 12: Pool size sensitivity (R@1 %). Strategy rankings are 100% stable across pool sizes spanning an order of magnitude.

Pool	Encoder	random	embedding	dataflow	emb−rand (pp)	Ranking
50	Qwen	54.5	56.5	55.5	+2.0	E>D>R
100	Qwen	42.8	44.7	43.8	+1.9	E>D>R
200	Qwen	34.2	35.9	35.1	+1.7	E>D>R
500	Qwen	26.1	27.6	26.9	+1.5	E>D>R
50	StarCoder	46.0	47.5	45.8	+1.5	E>R>D
100	StarCoder	35.8	37.2	35.7	+1.4	E>R>D
200	StarCoder	28.2	29.5	28.1	+1.3	E>R≈D
500	StarCoder	21.4	22.5	21.3	+1.1	E>R≈D

Table 13: Static vs dynamic embedding mining. Static mining (one-time, from pretrained) is sufficient.

Encoder	Static R@1 (%)	Dynamic R@1 (%)	Δ (pp)	p-value
Qwen	56.5	56.8	+0.3	0.41
StarCoder	47.5	47.7	+0.2	0.53

4.6.7 Weighted vs Binary Dataflow Negatives

Table 14: Weighted (by hop distance) vs binary dataflow negatives. Binary is sufficient.

Method	Qwen R@1 (%)	StarCoder R@1 (%)	Δ vs Binary (pp)
Binary (default)	55.5	45.8	—
Distance-weighted	55.8	45.9	+0.3 / +0.1

Weighting dataflow negatives by hop distance (farther = higher sampling probability) provides no significant improvement over binary connected/disconnected classification (+0.3pp Qwen, +0.1pp StarCoder; Table 14). The binary signal—"on the call chain or not"—captures the essential structural distinction. Further distance-based refinement adds complexity without benefit.

4.7 Failure Analysis

We manually analyzed 50 failure cases from the random retriever on SWE-bench Lite (Table 15). 44% (22/50) are retrieval failures (wrong files retrieved), of which 64% (14/22) involve cross-module dependencies where the gold file is a caller/callee of a retrieved-but-wrong file. These are precisely the cases dataflow training targets. 30% (15/50) are retrieval successes where the agent fails to generate correct patches, establishing a ceiling: even with perfect retrieval, ∼85% is the maximum achievable resolve rate given agent limitations.

Illustrative Example. Consider a Django issue about `QuerySet.filter()` returning wrong results with nested Q objects. The random retriever ranks `django/db/models/query.py` first (text match on "filter"), but the bug resides in `django/db/models/sql/query.py` (the downstream SQL builder invoked by `filter`). The dataflow-trained retriever correctly ranks the SQL builder file

Table 15: Failure category breakdown (50 random-retriever failures from SWE-bench Lite).

Category	Count	% of 50	Dataflow Could Help?
Retrieval: wrong files	22	44%	14/22 (64%)
Retrieval OK, agent fail	15	30%	N/A
Context window limit	7	14%	5/7 partially
Test env issue	4	8%	No
Ambiguous issue	2	4%	No

higher, having learned that structurally connected files along the call chain are more likely to co-occur in patches.

5 Discussion

Why Does the Encoder–Strategy Interaction Exist? Qwen2.5-Coder-3B exhibits representation saturation (mean pos_sim = 0.9855), compressing positive and negative embeddings into a narrow similarity band. Random negatives are trivially separable and provide weak gradient signal. Embedding mining selects negatives that are genuinely close in this compressed space, providing the difficulty gradient the encoder needs. Dataflow negatives add an orthogonal structural dimension that further assists discrimination.

StarCoder2-3B has healthier separation (mean pos_sim = 0.7985). Random negatives already provide moderate difficulty, and embedding mining further refines the boundary. Dataflow negatives offer no additional benefit because the embedding space is not saturated—the encoder can already separate structural from textual similarity using learned features alone.

Relationship to Inference-Time Graph Methods. StructFuse operates at training time, distilling structural knowledge into encoder representations via negative mining. Inference-time graph methods (CodexGraph, LocAgent, RANGER, ARISE, and others; Table 16) navigate or re-rank over code graphs at inference time. The two approaches are complementary: StructFuse produces a better base retriever that graph-based methods can navigate over. We do not claim superiority over these methods, as they solve different problems and operate under different computational budgets.

Table 16: Comparison with inference-time graph methods. StructFuse is the only method that leverages structural signal at training time without inference-time overhead.

Method	Graph Stage	Graph Type	Complementary?
StructFuse (ours)	Training	Dataflow (CodeQL)	—
CodexGraph	Inference	Code graph DB	Yes
LocAgent	Inference	Repo structure	Yes
RANGER	Inference	Call graph	Yes
ARISE	Inference	Intra-proc dataflow	Partially

Practical Recommendations. Based on our findings: (1) Always use true embedding-distance mining over random negatives—the improvement is consistent across encoders, benchmarks, temperatures, pool sizes, and data scales. (2) Never use BM25 negatives for code retrieval contrastive train-

ing. (3) Consider dataflow negatives when encoder representations are saturated (check pos_sim distribution); static mining suffices. (4) Binary structural signal is sufficient; weighted variants add complexity without benefit.

Limitations. Our study is limited to two 3B-parameter decoder-only code encoders; generalization to encoder-only architectures (e.g., CodeBERT) and larger models remains to be verified. CodeQL structural extraction achieves 34% coverage, limiting the effective reach of dataflow negatives. SWE-bench Lite evaluation uses 300 instances with a single agent model, providing limited statistical power for pairwise comparisons between mining strategies.

6 Conclusion

We presented a systematic study of hard negative mining strategies for contrastive code retrieval. Our controlled comparison of four strategies—BM25, random, dataflow, and embedding-distance—across two encoders, three benchmarks, and extensive ablations establishes three findings: (1) True embedding-distance mining consistently improves code retrieval, correcting the prior misconception that embedding negatives are harmful (the reported harm stems from BM25-proxy negatives). (2) Structure-grounded dataflow negatives provide conditional benefit when encoder representations are saturated, with the true per-instance effect (+2.8pp) masked by coverage-driven dilution in pooled results. (3) Negative difficulty quartile analysis provides mechanistic explanation: harder instances benefit more from better negatives (Q4: +3.2pp vs Q1: +0.8pp), linking mining strategy effectiveness to representation geometry.

These findings transfer to downstream tasks (SWE-bench Lite: +2.5pp embedding, +3.2pp dataflow resolve rate) and are robust across temperatures, pool sizes, data scales, and a second benchmark (RepoBench-R). Strategy rankings never reverse across any experimental condition tested.